Homework Requirements:

PART 1: EEG Brain Signal Classification

Mission Background

In this part, you will build machine learning pipelines to classify four motor-execution classes from EEG signals:

- 0: left hand
- 1: right hand
- 2: feet
- 3: rest

Part 1 is divided into two tasks with different generalization settings:

Task	**Setup**	**Main challenge**
Task 1	Train and test on the same subject	How band selection affects performance
Task 2	Train on multiple subjects, test on unseen subject data	How to generalize across subjects

Starter code is intentionally incomplete for preprocessing and model design. You are expected to implement and document the missing parts.

Part 1 is designed as an open implementation task:

- Task 1 requires your preprocessing and band-comparison experiments.
- Task 2 requires your own training design, checkpoint definition, and inference implementation.
- You have flexibility in methods, but required interfaces and output formats are mandatory.

---

1. Rules and Notes

- Use relative paths only. Do not hard-code absolute paths.

- Run Task 1 notebook from `part1/task1` and Task 2 scripts from `part1/task2`.
- Keep your code reproducible in a clean grading environment.
- Do not use official test labels for model tuning.
- Follow command-line and CSV format requirements exactly.
- You may use common libraries such as numpy, scipy, scikit-learn, pytorch, mne, and braindecode.
- You must not use external datasets.
- You must not use pretrained models.

Violation of interface or format requirements may cause execution failure in grading.

Quick Path Checks

Before running training/inference, verify required files from each task directory:

- Task 1 (run in `part1/task1`):

```
None
python -c "from pathlib import Path; req=[Path('data/train.npz'),
Path('data/test.npz')]; missing=[str(p) for p in req if not p.exists()];
print('OK' if not missing else f'Missing: {missing}')"
```

- Task 2 (run in `part1/task2`):

```
None
python -c "from pathlib import Path; req=[Path('data/test.npz')] +
[Path(f'data/train/subject{i:02d}.npz') for i in range(1,11)]; missing=[str(p) for
p in req if not p.exists()]; print('OK' if not missing else f'Missing:
{missing}')"
```

---

2. Dataset

The released dataset follows this layout:

```
None
part1/
  task1/
    data/
      train.npz
      test.npz
```

```
task2/
  data/
    train/
      subject01.npz
      subject02.npz
      ...
      subject10.npz
    test.npz
```

Required Arrays

Task 1 file:

- part1/task1/data/train.npz must provide: x, y
- part1/task1/data/test.npz must provide: x

Task 2 training files:

- each part1/task2/data/train/subjectXX.npz must provide: x, y

Task 2 test file:

- part1/task2/data/test.npz must provide: x

Tensor Convention

- one trial: (channels, time_points)
- batched array: (num_trials, channels, time_points)

---

3. Environment

Your final submission must run on a fresh environment with declared dependencies.

Recommended checks before submission:

- verify imports in a clean environment
- verify file paths from each task working directory
- verify that inference command runs end-to-end

---

4. Task 1: Within-Subject Classification

Starter file:

- part1/task1/task1.ipynb

Description

Task 1 focuses on one subject with a released fixed split. The core objective is to compare the impact of frequency-band choices on classification quality.

Required frequency bands:

- 8-13 Hz
- 13-30 Hz
- 4-40 Hz
- 70-125 Hz

What You Must Do

- implement preprocessing
- run full pipeline for all four required bands
- compare outcomes across all four runs
- report and explain your observations

What You Can Modify (Task 1)

Required:

- preprocessing pipeline
- training hyperparameters

Optional (Bonus):

- model selection
- feature extraction
- class balancing
- data augmentation
- other justified design changes

Split Policy (Strict)

- use the released fixed split in data/train.npz and data/test.npz when running inside `part1/task1`

---

5. Task 2: Cross-Subject Classification

Starter file:

- part1/task2/inference.py

Description

Task 2 is cross-subject classification. You train from released multi-subject training data and infer labels on unseen-subject test data.

Dataset Placement (Task 2)

For Task 2, dataset files must be placed under `part1/task2/data/` with the following fixed locations:

- training subjects: `part1/task2/data/train/subject01.npz` to `part1/task2/data/train/subject10.npz`
- testing set: `part1/task2/data/test.npz`

Do not move these files to other directories.

What You Must Do

- design your own training pipeline
- define and document your checkpoint format
- implement checkpoint loading and model reconstruction
- ensure inference reproduces the preprocessing expected by your checkpoint
- provide a runnable training command file: part1/task2/train_command.txt
- provide a Task 2 dependency file: part1/task2/requirements.txt

The training command file must contain one complete command (with all required parameters) that can be executed directly from `part1/task2`.

Execute `train_command.txt` from `part1/task2`.

Example (put this style of command in `part1/task2/train_command.txt`):

```
None
python train.py --data data/train --output-dir outputs/task2_train --checkpoint
outputs/task2_train/task2_model.pt --epochs 100 --batch-size 64 --learning-rate
0.001 --weight-decay 0.0001 --seed 42 --num-workers 0 --device auto
```

No complete Task 2 training pipeline is provided in starter materials.

Task 2 Requirements File

You must include `part1/task2/requirements.txt` so TAs can reproduce your Task 2 environment.

Do not use a raw `pip freeze` output as your final file. It often captures many unrelated packages and machine-specific CUDA bindings.

Recommended simple workflow:

1. Keep imports in Task 2 code explicit and minimal.
2. Generate an initial dependency list with one of the following:
- `pip-chill > requirements.txt`
- `pipreqs . --force --savepath requirements.txt`
3. Manually review `requirements.txt` and remove unrelated packages.
4. Prefer portable package names/version ranges (for example, avoid pinning local CUDA-specific wheel variants unless absolutely required).
5. In a clean environment, run:
- `pip install -r requirements.txt`
- your training/inference commands from `part1/task2` and confirm they work end-to-end.

Required CLI

Your script must run as:

python inference.py --data <data_dir_or_test_file> --checkpoint [--output]

Rules:

- --data can point to either:
    - task2 data directory containing test.npz (recommended: data)
    - test.npz directly (recommended: data/test.npz)
- --output is optional

Task 2 Submission CSV Format

Your Task 2 output file must satisfy all requirements below:

- default filename is submission.csv when --output is not given
- header must be exactly: id,label
- id must preserve dataset order
- label must be an integer in {0, 1, 2, 3}

Example:

```
None
id,label
0,2
1,0
2,3
```

---

6. Deliverables

Include the following in your Part 1 submission:

- the report PDF as a separate file from the code archive
- the code archive ZIP as a separate file from the report PDF
- part1/task1/task1.ipynb
- part1/task2/inference.py
- part1/task2/train_command.txt
- part1/task2/requirements.txt
- additional Task 2 files needed by your implementation

You should only submit **ONE report** for the entire homework. Including part 1 and 2.

Entire project submission layout:

```
None
{515512|515513}_{groupNum}_HW3_code.zip
  └ part1/
    ├ task1/
    |  └ task1.ipynb
    └ task2/
      ├ inference.py
      ├ train_command.txt
      ├ requirements.txt
      └ (additional Task 2 files)
 └ part2/
       └ part2.ipynb

{515512|515513}_{groupNum}_HW3_report.pdf
```

Naming:

- the code archive and report PDF must be named {515512|513}_group{Num}_HW3_{code|report}.{zip|pdf}, where {515512|513} is your course number
- example: group 99 from Mr. Wei's course should submit 515513_group99_HW3_code.zip and 515513_group99_HW3_report.pdf

Do not include:
- dataset files under part1/task1/data or part1/task2/data
- virtual environment folders
- cache folders

Notice : Incorrect file formats will result in **a score of 0** for this homework.

---

7. Report Requirements

Please write report for Part 1 in English or Chinese and include the following sections:

1. Method
2. Preprocessing Design
3. Experimental Results
4. Analysis / Discussion

At the beginning of the report, please include:
- group number
- all members' names and student IDs
- a simple peer evaluation table

For the peer evaluation table, each group may rate member contributions on a 1-10 scale. You may decide the score based on your own group collaboration, but the table should briefly indicate each member's contribution.

Task 1 Requirements

- compare at least two preprocessing designs
- explain which design works better and why

Task 2 Requirements

- explain how you address cross-subject generalization
- analyze which methods that work in within-subject setting may not transfer well to cross-subject setting
- explain the relationship between your local validation performance and public leaderboard performance

Task 1 Optional Bonus

- make and discuss at least one meaningful attempt beyond preprocessing
- examples: different model families, feature extraction, class balancing, augmentation, or other justified design changes

Task 2 Optional Bonus

- make and discuss at least one meaningful attempt beyond the basic cross-subject pipeline

Report Grading Rubric

The report will be graded with the following reference standard:

Report quality	**Reference score**
Excellent: very well written, complete, and shows strong or original insights	100%
Very good: well written, complete, and shows solid understanding and analysis	80%
Good: covers most required content with reasonable analysis, but lacks depth or clarity in some parts	60%
Fair: covers only part of the required content, with limited analysis or weak organization	40%
Poor: major required content is missing, and discussion is unclear or mostly descriptive	20%
Very poor / incomplete: report is largely missing, seriously incomplete, or not meaningful for evaluation	0%

---

8. Evaluation and Grading Mechanism

Reproducibility Gate (Mandatory)

Before leaderboard-based scoring is applied, submitted code must be runnable by TAs in a clean environment.

Hidden evaluation scripts may check:

- file format compliance

- whether code runs end-to-end in a clean environment
- whether the submission file can be reproduced
- runtime constraints (if announced)

Important policy:

- if a task implementation cannot be executed, that task implementation score is 0
- leaderboard score is only considered when code is reproducible

Score Components

- Task 1 implementation: 10%
- Task 2 implementation: 20%
- Report: 20% + 5%

Task 1 Implementation (10%)

Task 1 implementation uses a two-threshold plus interpolation policy based on leaderboard performance.

1. TA first verifies your submitted code is executable
2. TA baseline without preprocessing defines Threshold 1 on public leaderboard
3. TA baseline with basic preprocessing defines Threshold 2 on public leaderboard
4. Student score is mapped according to public/private leaderboard rules below

Scoring mapping:

- below Threshold 1:
    - linearly interpolated from the lowest valid public score to Threshold 1
    - falls in 10% x (0.4 to 0.6)
- at or above Threshold 1:
    - at least 10% x 0.6
- at or above Threshold 2:
    - at least 10% x 0.7
- above Threshold 2:
    - remaining 10% x 0.3 is assigned by private leaderboard rank (linear interpolation)

Task 2 Implementation (20%)

Task 2 implementation uses a single-threshold plus interpolation policy.

1. TA first verifies your submitted code is executable
2. TA baseline defines one public leaderboard threshold
3. Students below/above threshold are mapped to different score ranges

Scoring mapping:

- below threshold:
    - linearly interpolated from the lowest valid public score to threshold
    - falls in 20% x (0.4 to 0.6)
- at or above threshold:
    - at least 20% x 0.6
- above threshold:
    - remaining 20% x 0.4 is assigned by private leaderboard rank (linear interpolation)

Leaderboard Scoring Definitions

- valid public score: a Kaggle submission that is accepted and produces a valid score
- invalid submission, format error, or non-reproducible code is excluded from leaderboard scoring
- public leaderboard is used for threshold checks and below-threshold interpolation
- private leaderboard is used for ranking in above-threshold tiers

---

You can find all file you need for part 1 here :

- part 1 task1.ipynb
- part1 task2 inference.py
- part1 introduction
- part1 video
- part1 data
- part1 kaggle : announce at 4/28
    - task 1
    - task 2
- QA page